

Домашнее задание

Создание базовой аналитической витрины

В результате выполнения домашнего задания, студент подтвердит навыки:

1. Моделирования базовой аналитической витрины в корпоративном хранилище данных по представленному описанию предметной области;
2. Использования алгоритмов очистки данных с помощью языка SQL;
3. Реализации сложных SQL запросов, включающих обработку версионного джоина и сворачивание истории.

Для успешного выполнения домашнего задания необходимо реализовать прототип создания аналитической витрины на языке SQL. Прототип должен представлять собой несколько трансформов, которые могут быть использованы для построения ETL.

Итоговое решение приложить в формате *ДЗ_<Инициалы>_Скpунт.sql* к домашнему заданию в личном кабинете.

Описание предметной области

В качестве предметной области выбрана деятельность известной авиакомпании, которая осуществляет рейсовые авиамаршруты по России.

Каждый рейс (flights) следует из одного аэропорта (airports) в другой. Рейсы с одним номером имеют одинаковые пункты вылета и назначения, но будут отличаться датой отправления. Рейс обслуживает первый и второй пилот, а также группа бортпроводников (employee).

Данные были предварительно обработаны, денормализованы и поставлены в детальный слой корпоративного хранилища по схеме «звездочка»:

Таблица airports

Таблица – справочник, не имеет версий. Аэропорт идентифицируется трехбуквенным кодом (airport_code) и имеет свое имя (airport_name).

Для страны не предусмотрено отдельной сущности, но название (country) указывается и может служить для того, чтобы определить аэропорты одной страны.

DDL таблицы airports:

Столбец	Тип	Модификаторы	Описание
airport_code	char(3)	NOT NULL	Код аэропорта
airport_name	text	NOT NULL	Название аэропорта
country	text	NOT NULL	Город

Таблица employee

Таблица фактов содержит информацию о сотрудниках компании. Каждый сотрудник идентифицируется табельным номером (employee_id) и занимает должность (position) в соответствии с образованием:

- Pilot – пилот, имеет право на управление гражданским судном
- SFA – старший бортпроводник (senior flight attendant), ответственен за полетную документацию и взаимодействие с пилотами
- Steward – бортпроводник, ответственен за работу с пассажирами

Сотрудник содержится на ставке и получает почасовую оплату в рублях. При изменении атрибутов сотрудника создается новая версия строки. Primary Key таблицы сотрудников состоит из двух полей — табельный номер (employee_id) и дата открытия версии (valid_from_dt). Открытые версии имеют valid_to_dt равный '5999-01-01'.

DDL таблицы employee:

Столбец	Тип	Модификаторы	Описание
employee_id	int	NOT NULL	Табельный номер
employee_name	text	NOT NULL	Имя сотрудника
birth_dt	date	NOT NULL	Дата рождения
position	text	NOT NULL	Должность
salary	float	NOT NULL	Размер заработной платы в час
valid_from_dt	date	NOT NULL	Дата начала версии
valid_to_dt	date	NOT NULL	Дата конца версии

Таблица flights

Таблица измерений содержит информацию о рейсах. Primary Key таблицы рейсов состоит из трех полей — номера рейса (flight_no), табельного номера сотрудника (employee_id) и даты отправления (scheduled_departure).

Рейс всегда соединяет две точки — аэропорты вылета (departure_airport) и прибытия (arrival_airport). У каждого рейса есть запланированные дата и время вылета (scheduled_departure) и прибытия (scheduled_arrival). Реальные время вылета

(actual_departure) и прибытия (actual_arrival) могут отличаться: обычно не сильно, но иногда и на несколько часов, если рейс задержан.

Статус рейса (status) может принимать одно из следующих значений:

- Scheduled - Рейс доступен для бронирования. Это происходит за месяц до плановой даты вылета; до этого запись о рейсе не существует в базе данных.
- On Time - Рейс доступен для регистрации (за сутки до плановой даты вылета) и не задержан.
- Delayed - Рейс доступен для регистрации (за сутки до плановой даты вылета), но задержан.
- Departed - Самолет уже вылетел и находится в воздухе.
- Arrived - Самолет прибыл в пункт назначения.
- Cancelled - Рейс отменен.

DDL таблицы flights:

Столбец	Тип	Модификаторы	Описание
flight_no	char(6)	NOT NULL	Номер рейса
employee_id	int	NOT NULL	Табельный номер
scheduled_departure	timestamp	NOT NULL	Время вылета по расписанию
scheduled_arrival	timestamp	NOT NULL	Время прилёта по расписанию
departure_airport	char(3)	NOT NULL	Аэропорт отправления
arrival_airport	char(3)	NOT NULL	Аэропорт прибытия
status	varchar(20)	NOT NULL	Статус рейса
actual_departure	timestamp	NULL	Фактическое время вылета
actual_arrival	timestamp	NULL	Фактическое время прилёта

Задание

Вам нужно создать аналитическую витрину для расчета заработной платы сотрудников в рамках их должности и зарплатной ставки.

Выплаты производятся только за успешно совершенные рейсы. Расчет зарплаты производится по следующей формуле:

*кол-во часов в полете (actual_arrival - actual_departure) * ставку (salary) * коэффициент*

Коэффициент определяется как:

- 1 – для внутренних рейсов
- 1,2 – для международных рейсов

Тип рейса определяется через страну вылета и страну прибытия.

Важно! Если во время выполнения рейса сотруднику присвоена новая почасовая ставка – оплата производится по предыдущей ставке.

Итоговая аналитическая витрина должна иметь вид:

Столбец	Тип	Модификаторы	Описание
employee_id	int	NOT NULL	Табельный номер
position	text	NOT NULL	Должность
salary	float	NOT NULL	Размер заработной платы в час
flight_cnt	char(6)	NOT NULL	Количество выполненных рейсов
salary_gross	float	NOT NULL	Размер заработной платы за период
valid_from_dttm	date	NOT NULL	Дата начала периода
valid_to_dttm	date	NOT NULL	Дата конца периода

Демонстрационные данные

Данные для отладки скрипта можно найти в прикрепленном файле [script.sql](#). Файл содержит SQL-скрипт для базы данных PostgreSQL, создающий таблицы предметной области и наполняющий их данными.

Справочные материалы

Version Join

Алгоритм предназначен для соединения двух или более версионных таблиц с общим логическим ключом

Важно! Во всех входных таблицах должны присутствовать поля valid_from_dttm/dt и valid_to_dttm/dt

Реализация алгоритма состоит из трех шагов:

1. Создаем временную таблицу

```
create table output_TMP
(
    key1 <key1_column_data_type>
    ,key2 <key2_column_data_type>
    ...
    ,keyn <keyn_column_data_type>
    ,valid_from_dttm timestamp without time zone (valid_from_dt date)
    ,valid_to_dttm timestamp without time zone (valid_to_dt date)
) distributed by (key1 ... keyn )
;
```

2. Заполняем временную таблицу всевозможными версиями данных

```
insert into output_TMP
select  key1
      ,key2
      ...
      ,keyn
      ,valid_from_dttm/valid_from_dt
      ,lead(valid_from_dttm) over (partition by key1,...,keyn order by valid_from_dttm))
- interval '1 second' as valid_to_dttm
      ,lead(valid_from_dt) over (partition by key1,...,keyn order by valid_from_dt) -
interval '1 day' as valid_to_dt
from
(
  select  key1
        ,key2
        ...
        ,keyn
        ,valid_from_dttm
      from
      (
        select key1,...,keyn, valid_from_dttm/valid_from_dt from input_tbl_1
        union all
        select key1,...,keyn, (valid_to_dttm + interval '1
second' as valid_to_dttm)/(valid_to_dt + interval '1 day') from input_tbl_1
        union all
        select key1,...,keyn, valid_from_dttm/valid_from_dt from input_tbl_2
        union all
        select key1,...,keyn, (valid_to_dttm + interval '1
second' as valid_to_dttm)/(valid_to_dt + interval '1 day')  from input_tbl_2
        -- ...
        union all
        select key1,...,keyn, valid_from_dttm/valid_from_dt from input_tbl_n
        union all
        select key1,...,keyn, (valid_to_dttm + interval '1
second' as valid_to_dttm)/(valid_to_dt + interval '1 day') from input_tbl_n
      ) sq
      group by  key1,...,keyn
              ,valid_from_dttm/valid_from_dt
    ) sq;
```

3. Собираем итоговую таблицу

```
insert into output
select  tmp.key1,...,tmp.keyn
        ,tmp.valid_from_dttm/tmp.valid_from_dt
        ,tmp.valid_to_dttm/tmp.valid_to_dt
        ,t1.attr1 - t1.attrn
        ,t2.attr1 - t2.attrn
        -- ...
        ,tn.attr1 - tn.attrn
from output_TMP tmp
left join table1 t1
  on t1.key1 = tmp.key1
 and t1.key2 = tmp.key2
  ...
 and t1.valid_from_dttm <= tmp.valid_from_dttm
 and t1.valid_to_dttm   >= tmp.valid_to_dttm
left join table2 t2
  on t2.key1 = tmp.key1
 and t2.key2 = tmp.key2
  ...
 and t2.valid_from_dttm <= tmp.valid_from_dttm
 and t2.valid_to_dttm   >= tmp.valid_to_dttm
-- ...
left join tablen tn
  on tn.key1 = tmp.key1
 and tn.key2 = tmp.key2
  ...
 and tn.valid_from_dttm <= tmp.valid_from_dttm
 and tn.valid_to_dttm   >= tmp.valid_to_dttm
WHERE valid_to_dttm IS NOT NULL
;
```

Rollup History

Алгоритм, реализующий схлопывание версий записей в таблице (объединения идущих подряд версий с одинаковыми значениями атрибутов)

Важно! Во всех входных таблицах должны присутствовать поля valid_from_dttm/dt и valid_to_dttm/dt

Реализация алгоритма состоит из трех шагов:

1. Выбрать список полей из источника, по которым необходимо свернуть историю
2. Проставить флаг уникальной строки для выборки из п.1

```

SELECT H.*,
       CASE WHEN
         /*в первом блоке Row перечисляются все поля, которые участвуют в сравнении+
         добавить [,1] если нужно сохранить пустую строку keep_first_row_NULL=TRUE*/
         Row( H.key_2 ,H.fld_int ,H.fld_str ,H.fld_str_txt ,H.fld_dttm , 1 ) ---<<<
         добавить 1 если нужно сохрнить пустую строку
         IS DISTINCT FROM
         /*во втором блоке Row перечислить поля, которые попали в список первого Row +
         добавить Lag(1) OVER (W) если нужно сохранить пустую строку keep_first_row_NULL=TRUE*/
         Row( Lag(H.key_2) OVER (W)
              ,Lag(H.fld_int) OVER (W)
              ,Lag(H.fld_str) OVER (W)
              ,Lag(H.fld_str_txt) OVER (W)
              ,Lag(H.fld_dttm) OVER (W)
              ,Lag(1) OVER (W) ---<<< добавить если нужно сохрнить пустую строку
            )
       THEN 1 ELSE 0 END as chng_flg
       , Last_value(valid_to_dttm) OVER (W ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING) as last_dttm
FROM (/*input_table или зарос с протягиваемыми полями из п.1*/) AS H
WINDOW W AS (PARTITION BY H.<ключ, может быть составным> ORDER BY H.valid_from_dttm)

```

3. Собрать итоговую таблицу, восстановив конец версии

```

SELECT
  HF.key_1
, HF.key_2
, HF.fld_int
, HF.fld_str
, HF.fld_str_txt
, HF.fld_dttm
, HF.valid_from_dttm
, coalesce(Lead(HF.valid_from_dttm - INTERVAL '<INTERVAL, по умолчанию 1 second>') OVER
(PARTITION BY HF.<ключ, может быть составным> ORDER BY HF.valid_from_dttm)
, HF.last_dttm) as valid_to_dttm
FROM (запрос из п.2) AS HF
WHERE HF.chng_flg = 1;

```